N051 한 번 만든 이미지로 배포하기_학습서

Chapter01

한 번 만든 이미지로 배포하기

이 교안은 에이전트 서비스가 PR 머지 때 CI(지속적 통합, Continuous Integration. 코드를 합칠 때마다 테스트를 자동으로 돌리는 것)를 거쳐 staging과 production에 자동으로 배포되고, 실행 환경을 저장소 루트의 Dockerfile이 정하는 상태에서 시작합니다. 배포 단계는 코드를 Cloud Build로 보내고, Cloud Build가 그 Dockerfile로 이미지를 만듭니다. 이 교안에서도 지금까지 배운 요소로 만든 AI 친구 에이전트 서비스를 예로 들어 실습하고, 줄여서 에이전트 서비스라고 부르겠습니다.

배포가 자동으로 돌기 시작하면, 문제가 생긴 순간에 답해야 하는 질문이 생깁니다. 사용자가 "약속 알림이 이상해요"라고 알려 왔다면, 지금 production에서 도는 코드가 어느 커밋인지, 바로 전에 정상이던 버전으로 되돌릴 수 있는지, 지금 production이 staging에서 확인한 바로 그 이미지인지를 빨리 알아야 합니다. 오늘은 먼저 지금 구조로 이 세 질문에 답해 보고, 답하기 어려운 이유를 하나씩 없앱니다. 오늘은 이미지를 GitHub Actions 안에서 직접 만들고 커밋 SHA(커밋마다 붙는 고유한 해시값. 예: a1b2c3d)를 태그로 붙여, 커밋에서 실행 중인 리비전까지 한 줄로 이어지게 만듭니다. 이어서 staging에서 확인한 이미지를 다시 빌드하지 않고 그대로 production에 보내 보고, 쌓이는 이미지를 정리하는 규칙도 겁니다.

주관식 퀴즈

Q. [시작 질문] production에서 문제가 생겼다는 신고가 들어왔습니다. 지금 production의 코드가 어느 커밋인지, 정상이던 버전으로 되돌릴 수 있는지, staging에서 확인한 이미지와 같은지 세 가지를 알아내려면 어디를 어떤 순서로 열어 보겠습니까? 아는 화면을 떠올려 적어 보세요.

0/300**다음**

오늘의 학습 목표

1. 이미지 이름의 구조를 읽고, 태그와 다이제스트(digest, 이미지 내용으로 계산한 고유 해시값)가 어떻게 다른지 설명할 수 있습니다.
2. 커밋 SHA를 태그로 쓰는 이유를 설명하고, `latest` 만 쓸 때 생기는 문제를 짚을 수 있습니다.
3. GitHub Actions에서 이미지를 만들어 Artifact Registry에 올리고 그 이미지로 배포하는 워크플로를 실행 전 기준으로 검토할 수 있습니다.
4. 커밋, 이미지 태그, 다이제스트, 리비전, `/version` 응답을 콘솔과 명령으로 찾아 한 줄로 이을 수 있습니다.
5. staging에서 확인한 이미지를 다이제스트로 지정해 production에 배포하고, 두 환경의 다이제스트가 같은지 확인할 수 있습니다.
6. 로컬 환경(내 컴퓨터)에서 만든 이미지가 Cloud Run에서 실행되지 않는 이유를 설명하고 작업 규칙에 반영할 수 있습니다.
7. 정리 정책으로 저장 공간을 관리하되, 롤백(rollback, 문제가 생긴 새 버전 대신 이전 버전으로 되돌리는 것)에 필요한 이미지를 남기는 기준을 정할 수 있습니다.

8. 고장 난 Dockerfile을 증상과 기록만 보고 원인을 찾아 고칠 수 있습니다.

Chapter02

시작 상태 맞추기

이 교안의 실습은 아래 상태가 갖춰져 있어야 진행할 수 있습니다. 표의 마지막 줄인 루트의 Dockerfile 까지 있어야 오늘 GitHub Actions에서 그 Dockerfile로 이미지를 만들 수 있습니다. 표에 나오는 시크릿(secret)은 API 키처럼 밖에 드러나면 안 되는 값이고, Google Cloud 콘솔에서는 "보안 비밀"로 표시됩니다.

앞 실습을 이어서 하고 있다면 아래 확인 명령만 실행하고 넘어갑니다. 빠진 상태가 있으면 템플릿의 실습별 시작 상태 문서에서 "없으면" 칸의 항목을 표의 위 칸부터 차례로 실행합니다. 처음부터 모두 맞추면 한두 시간이 걸리므로, 빠진 항목이 많다면 수업 전에 맞춰 두거나 강사에게 먼저 알립니다.

필요한 상태	확인할 것	없으면
Google Cloud 프로젝트, gcloud 로그인과 기본 리전, API 4개	`gcloud config list` 에 내 프로젝트와 `run/region = us-central1`	A
`friend-agent` 서비스 (시크릿 `llm-api-key` 참조)	`gcloud run services list` 에 `friend-agent`	B
GitHub Actions 배포 권한(WIF)과 저장소 변수	`gh variable list` 에 `GCP_PROJECT_ID`, `WIF_PROVIDER`, `DEPLOY_SA`, `RUNTIME_SA`, `BUILD_SA`	`docs/wif-setup.md` 0~4, 7단계
CI(`ci.yml`)와 환경별 배포(`deploy.yml`), `staging` 브랜치, GitHub 환경 `staging`·`production`	`.github/workflows` 에 두 파일, 원격 `staging` 브랜치, 환경 두 개	C, D
`friend-agent-staging` 서비스	`gcloud run services list` 에 `friend-agent-staging`	E
저장소 루트의 `Dockerfile`, `.dockerignore`	두 파일이 루트에 있음	F

직접 실행할 명령: 시작 상태 확인

```
gcloud config list
gcloud run services list --region us-central1
gh variable list
gh api repos/{owner}/{repo}/environments --jq '.environments[].name'
git branch -r
ls .github/workflows Dockerfile .dockerignore
```

코딩 에이전트에게 맡길 때: 시작 상태 확인

```
docs/start-states.md 의 표를 보고, 이 저장소와 지금 gcloud 프로젝트가 '이미지 배포' 실습을 시작할 상태인지 확인해 줘.
확인 명령만 실행하고 아무것도 만들거나 바꾸지 마. 빠진 항목이 있으면 문서의 어느 항목(A~I)을 실행해야 하는지 알려 줘.
```

이미지를 구별하는 값, 다이제스트

오늘은 이미지 "다이제스트"라는 말이 계속 나옵니다. 다이제스트(digest)는 이미지를 올릴 때 이미지 내용으로 자동으로 정해지는 `sha256:…` 모양의 값으로, 이미지 하나를 정확히 가리키는 데 씁니다.

이런 값이 따로 있는 이유는 사람이 붙이는 이름인 태그가 믿을 만하지 않기 때문입니다. 태그는 언제든 다른 이미지로 옮겨 붙일 수 있어서, 어제의 `latest` 와 오늘의 `latest` 가 다른 이미지일 수 있습니다. 내용이 정하는 다이제스트는 옮겨 붙일 수 없으므로, "정확히 이 이미지"를 말해야 할 때는 다이제스트를 씁니다.

오늘 다이제스트를 확인하는 이유는 세 가지입니다. 리비전이 실제로 어떤 이미지로 실행 중인지 알기 위해, staging과 production이 같은 이미지를 실행하는지 비교하기 위해, 그리고 staging에서 확인한 바로 그 이미지를 production에 보낼 때 그 이미지를 지정하기 위해서입니다. Cloud Run은 리비전을 만들 때 태그로 받은 이미지도 그 순간의 다이제스트로 바꿔 기록하므로, 리비전 상세의 다이제스트가 가장 정확한 답입니다.

오늘은 아래 세 곳에서 다이제스트를 만납니다. 값이 길기 때문에 비교할 때는 앞 12자리 정도만 맞춰 봐도 충분합니다.

보이는 곳	모양
Artifact Registry 이미지 목록의 다이제스트 칸	`sha256:0d01a69ed26e…`
Cloud Run 리비전 상세의 이미지 주소	`…/friend-agent@sha256:0d01a69ed26e…` 처럼 이름 뒤에 `@` 와 함께 붙음
이미지를 올리는 `docker push` 의 마지막 줄	`<태그>: digest: sha256:0d01a69ed26e… size: …`

장애 대응 훈련

시작 질문의 상황을 실제로 해 봅니다. production 에이전트 서비스에서 약속 알림이 이상하다는 신고가 들어왔다고 가정하고, 타이머를 5분으로 맞춘 뒤 아래 세 질문에 답합니다. 아직 아무것도 고치지 않은 지금 구조 그대로, 콘솔과 GitHub 화면만으로 답을 찾습니다. 답을 찾지 못한 질문은 왜 찾지 못했는지를 적습니다.

질문	내 답	찾은 곳, 또는 찾지 못한 이유	걸린 시간
1. 지금 production에서 도는 코드는 어느 커밋인가			
2. 바로 전에 정상이던 버전은 어느 이미지이고, 그 이미지로 되돌릴 수 있는가			
3. 지금 production 이미지는 staging에서 확인한 그 이미지와 같은가			

직접 확인할 곳: 훈련에서 열어 볼 화면

1. Cloud Run > `friend-agent` > 리비전 탭: 트래픽을 받는 리비전과 바로 전 리비전, 각 리비전 상세의 컨테이너 탭에 적힌 이미지 주소
2. production 주소의 `/version` : `commit` 과 `revision` 값
3. Artifact Registry > `cloud-run-source-deploy` > `friend-agent` : 이미지마다 붙은 태그와 다이제스트, 만든 시각
4. GitHub 저장소 > Actions 탭: main 의 deploy 실행 기록과 실행 시각
5. Cloud Run > `friend-agent-staging` > 리비전 탭: 트래픽을 받는 리비전의 이미지 다이제스트

결과 해석: 지금 구조로 답하기 어려운 이유

1번은 `/version` 의 `commit` 으로 답할 수 있습니다. 배포 워크플로가 커밋 SHA를 환경변수로 넣어 두었기 때문입니다. 하지만 이 값은 지금 트래픽을 받는 리비전에 대해서만 알려 주고, 이미지 쪽에는 커밋이 적혀 있지 않습니다.

2번에서 막히기 시작합니다. 바로 전 리비전이 쓰던 이미지는 리비전 상세에서 다이제스트로만 보이고, Artifact Registry 목록의 이미지에는 태그가 거의 없습니다. 어느 이미지가 어느 커밋인지는 만든 시각을 Actions 실행 기록과 하나씩 맞춰 봐야 하고, 쌓인 이미지가 많을수록 오래 걸립니다. 그 이미지가 아직 저장소에 남아 있는지도 따로 확인해야 합니다.

3번의 답은 대개 "다르다"입니다. staging 리비전과 production 리비전의 다이제스트가 다르게 나오는데, 배포 워크플로가 staging에 머지할 때 한 번, main에 머지할 때 또 한 번 소스를 올려 이미지를 따로 만들기 때문입니다. 같은 커밋으로 같은 Dockerfile을 써도 따로 빌드한 이미지는 서로 다른 이미지입니다.

다른 이미지가 위험한 이유

3번이 실제로 문제가 되는 경우를 에이전트 서비스의 코드로 살펴봅니다. 템플릿 `requirements.txt` 에는 `anthropic>=1.15` , `fastapi>=0.115` 처럼 라이브러리가 "이 버전 이상"으로 적혀 있습니다. 이렇게 적으면 이미지를 빌드할 때마다 그 순간의 최신 버전을 내려받습니다. 월요일에 staging에서 확인하고 수요일에 main에 머지했는데 그사이 라이브러리 새 버전이 나왔다면, 수요일에 다시 빌드한 production 이미지에만 새 버전이 들어갑니다. staging에서는 멀쩡했던 기능이 production에서만 깨지는 일이 이렇게 생기고, 이때 staging에서 한 확인은 production 이미지에 대해서는 아무것도 보장하지 않습니다. Dockerfile의 베이스 이미지와 `apt-get install` 로 받는 시스템 패키지도 같은 이유로 빌드할 때마다 달라질 수 있습니다.

그래서 확인한 것과 내보낸 것을 같게 만들려면, 이미지를 한 번만 만들어 이름을 붙여 두고 그 이미지를 staging과 production에 그대로 보내야 합니다. 한 번 만든 이미지를 여러 환경에 그대로 보내는 원칙을 build once, deploy many라고 부릅니다.

오늘 해 볼 일

오늘 할 일은 훈련의 세 질문에 몇 초 만에 답할 수 있는 구조를 만드는 것입니다. 섹션마다 어느 질문을 푸는지 정리하면 아래와 같습니다.

훈련 질문	지금 막히는 이유	해결하는 섹션
1. 지금 production은 어느 커밋인가	이미지에 커밋이 적혀 있지 않음	섹션 3: Actions에서 이미지를 만들고 커밋 SHA 태그를 붙임
2. 정상이던 버전으로 되돌릴 수 있는가	태그 없는 이미지를 시각으로 맞춰 찾아야 함. 지워졌을 수도 있음	섹션 4: 커밋, 이미지, 리비전을 잇는 연결표. 섹션 5: 롤백할 이미지를 남기는 정리 정책
3. staging에서 확인한 이미지와 같은가	환경마다 이미지를 따로 빌드함	섹션 4: staging 이미지를 그대로 production에 보내는 이미지 승격. 명령으로 해 보고 배포 워크플로로 옮김

섹션 6에서는 고장 난 Dockerfile을 고치며 이미지를 다뤄 보고, 섹션 7에서 같은 훈련을 한 번 더 해 걸린 시간을 비교합니다. 먼저 훈련에서 열어 본 이미지 목록과 이미지 이름을 자세히 봅니다.

소스 배포로 쌓인 이미지 보기

훈련의 2번에서 막혔던 이미지 목록을 다시 엽니다. 시작 상태의 배포 단계는 코드를 Cloud Build로 보내 이미지를 만들고, 만들어진 이미지는 Artifact Registry의 `cloud-run-source-deploy` 저장소에 저장됩니다. 배포할 때마다 이미지가 하나씩 생기므로, 앞 실습을 이어서 했다면 여러 개가, 시작 상태를 방금 맞췄다면 한두 개만 있을 수 있습니다. 먼저 이 저장소에 무엇이 얼마나 쌓였는지 봅니다. 목록에서 볼 것은 세 가지입니다. 이미지가 몇 개인지, 각 이미지에 어떤 태그가 붙어 있는지, 저장소 전체 크기가 얼마인지입니다.

직접 실행할 명령: 소스 배포 이미지 목록과 저장소 크기

```
# 이미지 목록. 태그와 만든 시각이 함께 나온다
gcloud artifacts docker images list us-central1-docker.pkg.dev/<PROJECT_ID>/cloud-run-source-deploy --in
clude-tags

# 저장소 정보(크기 포함)
gcloud artifacts repositories describe cloud-run-source-deploy --location us-central1
```

코딩 에이전트에게 맡길 때: 쌓인 이미지 보기

```
Artifact Registry의 us-central1 cloud-run-source-deploy 저장소에 있는 이미지를 최근 것부터 태그와 함께 보여 주고,
저장소 전체 크기도 알려 줘. 조회만 하고 아무것도 지우거나 바꾸지 마.
```

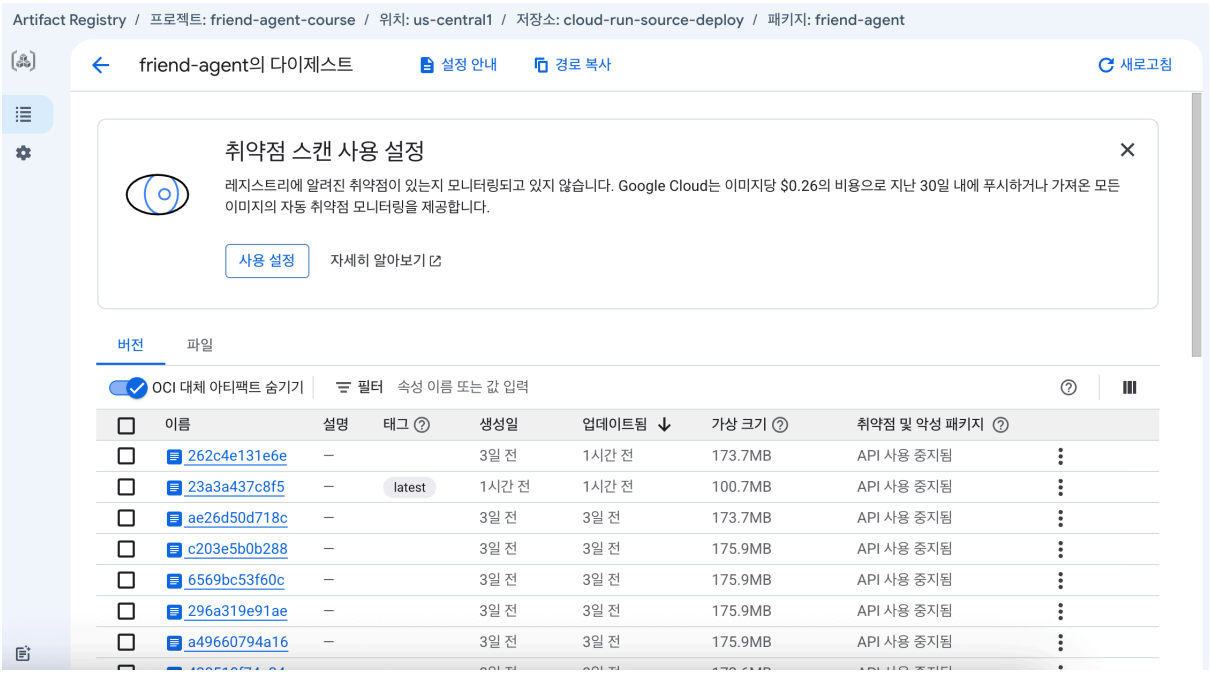


그림: Artifact Registry > 저장소(Repositories) > cloud-run-source-deploy > friend-agent: 배포 횟수만큼 쌓인 이미지 목록, 각 이미지의 태그와 크기, 만든 시각

이미지가 한두 개뿐이어도 볼 것은 같습니다. 이미지는 배포한 횟수만큼 생기지만, 이름이나 태그에 커밋 SHA가 들어 있지 않아 "이 이미지가 어느 커밋인가"는 만든 시각과 Actions 실행 기록을 맞춰 보며 짐작해야 합니다. 크기도 확인합니다. tesseract와 한국어 데이터가 들어간 뒤로 이미지 하나가 커졌고, Artifact Registry 무료 저장 공간은 계정당 월 0.5GB입니다(2026년 10월 기준). 그보다 많이 쓰면 GB당 월 0.1달러 정도의 요금이 붙습니다.

이미지 이름의 구조

Artifact Registry에 저장하는 이미지는 모두 같은 형식의 이름을 씁니다. 슬래시로 나뉜 네 부분과 콜론 뒤의 태그로, 앞에서부터 리전 주소, 프로젝트, 저장소, 이미지 이름, 태그입니다. 지금까지 소스 배포가 만든 이미지도 이 형식으로 저장되어 왔고, 오늘은 같은 형식에서 저장소와 태그를 직접 정합니다. 두 이름을 나란히 놓으면 차이가 보입니다.

	이미지 이름	저장소를 정한 쪽	태그
지금까지 (소스 배포)	`us-central1-docker.pkg.dev/<프로젝트 ID>/cloud-run-source-deploy/friend-agent`	소스 배포가 자동으로 만듦	대부분 비어 있음. 다이제스트로만 구별
오늘부터 (Actions 빌드)	`us-central1-docker.pkg.dev/<프로젝트 ID>/friend-agent/friend-agent:<커밋 SHA>`	내가 만듦	커밋 SHA

이미지 이름은 이미지가 저장된 주소이자, 여러 도구가 같은 이미지를 가리킬 때 쓰는 이름입니다. 이미지를 올리는 쪽, 실행하는 쪽, 정리하는 쪽이 모두 이 이름으로 같은 이미지를 찾습니다. 오늘 실습에서는 아래 네 곳에서 이 이름을 만납니다.

쓰이는 곳	이름으로 하는 일	오늘 만나는 자리
이미지 올리기 (`docker push`)	이 이름의 저장소와 태그로 이미지를 올린다	섹션 3 배포 워크플로의 이미지 만들기 단계
배포 (`--image` , 워크플로의 `image:`)	Cloud Run이 이 이름의 이미지를 받아 리비전을 만든다	섹션 3 워크플로, 섹션 4 이미지 승격
리비전 상세	리비전마다 어떤 이미지로 실행 중인지 이 이름과 다이제스트로 남는다	섹션 4 연결표
정리 정책	어느 저장소의 어떤 태그를 남기고 지울지 이 이름을 기준으로 정한다	섹션 5 저장 공간 관리

그래서 이름을 읽을 줄 알아야 워크플로 파일의 이미지 줄이 맞는지 검토하고, 리비전 상세를 보고 지금 production에서 무엇이 실행 중인지 답하고, 정리 정책이 롤백에 필요한 이미지를 지우지 않는지 판단할 수 있습니다. 이름에 커밋 SHA가 들어 있으면 이 네 곳 어디에서 이미지를 만나도 바로 GitHub의 커밋으로 거슬러 올라갈 수 있습니다.

위에서 본 소스 배포 이미지 목록에 태그가 거의 없었던 것도 이 때문입니다. 이름만으로는 이미지끼리 구별되지 않아, 어느 커밋으로 만든 이미지인지 알려면 만든 시각을 기록과 맞춰 봐야 했습니다. 아래 그림은 오늘부터 쓸 이름을 부분별로 나눈 것입니다.



그림: 이미지 이름의 네 부분과 태그, 그리고 태그와 다이제스트의 차이

앞의 세 부분은 이미지가 어디에 있는지를 말합니다. 리전과 Artifact Registry 주소, 내 프로젝트, 그 안의 저장소입니다. 네 번째가 이미지 이름이고, 콜론 뒤의 태그가 같은 이름의 이미지 여러 개를 구별합니다. 태그는 사람이 붙이는 이름이라서 다른 이미지로 옮겨 붙일 수 있습니다. 반면 앞에서 본 다이제스트는 이미지 내용이 정하는 값이라 옮겨 붙일 수 없습니다. 이미지 이름 뒤에 태그 대신 `@sha256:…` 을 붙이면 그 다이제스트의 이미지 하나만 정확히 가리킵니다.

가장 흔히 쓰는 태그는 `latest` 입니다. 새 이미지를 올릴 때마다 `latest` 가 새 이미지로 옮겨 가므로, 어제 배포한 `latest` 와 오늘의 `latest` 는 다른 이미지일 수 있고 이름만 봐서는 어떤 코드인지 알 수 없습니다. 그래서 오늘부터는 커밋 SHA를 태그로 씁니다. 태그를 보면 GitHub에서 그 커밋을 바로 찾을 수 있고, 커밋마다

이미지가 하나씩 생깁니다. 다만 같은 커밋 SHA 태그로 다른 이미지를 다시 올리지 않는다는 약속은 지켜야 하고, 실제로 어떤 내용이 실행 중인지 끝까지 확인할 때는 다이제스트를 봅니다.

객관식 퀴즈

Q. 어제 `friend-agent:latest` 로 배포했고, 오늘 다른 코드로 이미지를 만들어 같은 `latest` 태그로 올렸습니다. 어제 배포한 리비전에 대한 설명으로 옳은 것은 무엇인가요?

• 1. 어제 리비전도 자동으로 오늘 이미지로 바뀐다
• 2. 어제 리비전은 배포할 때 고정된 다이제스트의 이미지로 계속 실행되지만, 태그만 봐서는 어제와 오늘 이미지를 구별할 수 없다
• 3. 같은 태그로 올리면 Artifact Registry가 거부한다
• 4. 태그가 같으므로 두 이미지의 다이제스트도 같다

- •

  1. 어제 리비전도 자동으로 오늘 이미지로 바뀐다

- •

  2. 어제 리비전은 배포할 때 고정된 다이제스트의 이미지로 계속 실행되지만, 태그만 봐서는 어제와 오늘 이미지를 구별할 수 없다

- •

  3. 같은 태그로 올리면 Artifact Registry가 거부한다

- •

  4. 태그가 같으므로 두 이미지의 다이제스트도 같다

Chapter03

바뀌는 흐름

훈련의 1번 질문부터 풉니다. 이 섹션에서는 이미지를 만드는 곳을 Cloud Build에서 GitHub Actions 러너로 옮기고, 이미지 이름에 커밋 SHA를 남깁니다. 머지하면 배포된다는 흐름은 그대로이고, 아래 그림의 위아래 두 줄처럼 이미지를 누가 만들고, 어느 저장소에, 어떤 이름으로 두는지가 바뀝니다. 지금까지는 Actions가 소스 코드를 올리면 Cloud Build가 이미지를 만들어 자동 저장소 `cloud-run-source-deploy` 에 태그 없이 두었습니다. 오늘부터는 Actions 러너가 `docker build` 로 이미지를 만들고, 커밋 SHA 태그를 붙여 내가 만든 저장소 `friend-agent` 에 `docker push` 로 올린 다음, Cloud Run에는 소스 대신 그 이미지 이름을 넘깁니다.



그림: 소스 배포와 Actions 이미지 빌드 배포의 단계 비교, 처음 한 번 준비할 것과 머지한 뒤 확인할 것

그림 아래 왼쪽 칸이 이 섹션에서 처음 한 번만 하는 준비이고, 오른쪽 칸이 준비를 마치고 머지한 뒤 확인할 것입니다. 준비 1~3은 바로 아래 "이미지 저장소와 쓰기 권한"에서, 준비 4는 "워크플로 바꾸기"에서 하고, 확인은 "staging에서 확인하고 production으로"에서 합니다.

이미지 저장소와 쓰기 권한

그림의 준비 1~3입니다. Actions 러너가 만든 이미지를 올리려면 그 이미지를 받을 저장소가 있어야 하고, 배포 서비스 계정(배포 작업에만 쓰는 서비스 계정) `github-deployer` 가 그 저장소에 이미지를 올릴 권한을 가져야 합니다. 템플릿의 `docs/wif-setup.md` 5단계가 이 일을 합니다. 권한은 프로젝트 전체가 아니라 저장소 `friend-agent` 하나에만 줍니다.

직접 실행할 명령: 저장소 만들기와 쓰기 권한 주기

```
# 이미지를 받을 Docker 저장소
gcloud artifacts repositories create friend-agent --project <PROJECT_ID> --repository-format docker --lo
cation us-central1

# 배포 서비스 계정에 이 저장소 하나에만 쓰기 권한
gcloud artifacts repositories add-iam-policy-binding friend-agent --project <PROJECT_ID> --location us-c
entral1 --member "serviceAccount:github-deployer@<PROJECT_ID>.iam.gserviceaccount.com" --role roles/arti
factregistry.writer

# 워크플로가 읽을 저장소 주소
gh variable set AR_REPO --body "us-central1-docker.pkg.dev/<PROJECT_ID>/friend-agent"
```

코딩 에이전트에게 맡길 때: 저장소와 권한

```
docs/wif-setup.md 의 5단계를 따라 us-central1에 Docker 저장소 friend-agent 를 만들고,
github-deployer 서비스 계정에 이 저장소 하나에 대해서만 roles/artifactregistry.writer 를 줘.
그다음 GitHub 저장소 Variables 에 AR_REPO 를 us-central1-docker.pkg.dev/<프로젝트 ID>/friend-agent 로 넣어 줘.
명령을 실행하기 전에 그 명령이 무엇을 만들고 누구에게 어떤 권한을 주는지 한 줄씩 설명해 줘.
```

권한 명령에서는 두 가지를 봅니다. 명령이 `projects add-iam-policy-binding` 이 아니라 `repositories add-iam-policy-binding` 인지(대상이 저장소 하나인지), 역할이 `artifactregistry.writer` 인지(저장소 관리자 같은 넓은 역할이 아닌지)입니다. 에이전트가 오류를 피하려고 프로젝트 전체에 권한을 주자고 하면 거절하고 오류 메시지를 먼저 보여 달라고 합니다.

워크플로 바꾸기

그림의 준비 4입니다. 템플릿의 `starter/workflows/deploy-image.yml` 이 이미지 기반 배포 워크플로입니다. 시작 상태의 `.github/workflows/deploy.yml` 과 비교하면 테스트를 먼저 돌리고, staging과 production을 나누고, production은 승인을 거치는 구조는 같습니다. 달라지는 것은 배포 작업 안의 단계입니다. `setup-gcloud` 로 gcloud를 준비하고, `docker build` 로 이미지를 만들어 `docker push` 로 올린 뒤, `deploy-cloudrun` 에 `source: .` 대신 `image:` 로 그 이미지를 넘깁니다. 이미지 이름은 `${{ vars.AR_REPO }}/friend-agent:${{ github.sha }}` 이고, `github.sha` 가 이번 실행의 커밋 SHA입니다. 배포할 때 `COMMIT_SHA` 환경변수에도 같은 값을 넣으므로 `/version` 응답에 커밋 SHA가 나옵니다.

직접 실행할 명령: 워크플로 교체를 PR로 올리기

```
git switch -c image-deploy
cp starter/workflows/deploy-image.yml .github/workflows/deploy.yml
git diff
git add .github/workflows/deploy.yml
git commit -m "Actions에서 만든 이미지로 배포"
git push -u origin image-deploy
gh pr create --base staging --title "Actions에서 만든 이미지로 배포" --body "이미지를 커밋 SHA 태그로 만들어 배포"
```

코딩 에이전트에게 맡길 때: 워크플로 교체

```
image-deploy 브랜치를 만들고, starter/workflows/deploy-image.yml 내용으로 .github/workflows/deploy.yml 을 바꿔
줘.
액션 버전과 테스트, 환경, 승인 구조는 바꾸지 마. 바꾼 부분을 git diff 로 보여 주고 내가 확인하면
커밋해서 staging 으로 PR을 올려 줘.
```

PR을 올리기 전에 아래 기준으로 차이(diff)를 읽습니다. 기준에 걸리는 변경이 있으면 머지하지 않고 고쳐 달라고 합니다.

확인할 것	걸리면
Artifact Registry에 올리는 권한만 추가했는가 (저장소 관리자 같은 넓은 역할이 아닌가)	템플릿 역할 목록(`docs/wif-setup.md`)과 대조
이미지 태그에 커밋 SHA(`github.sha`)를 쓰는가 (`latest` 만 쓰지 않는가)	수정 요청
이미지 빌드 단계에 API 키 같은 시크릿을 넘기는가	거부. 키는 실행할 때 Secret Manager에서 읽는다
액션 버전이 템플릿과 같은가 (`checkout@v7` , `auth@v3` , `setup-gcloud@v3` , `deploy-cloudrun@v3`)	거부. 템플릿 버전 유지

staging에서 확인하고 production으로

준비를 마쳤으면 그림 오른쪽 아래 칸의 확인을 합니다. PR에서 CI가 통과하면 staging으로 머지합니다. 배포 워크플로가 실행되면 GitHub 저장소의 Actions 탭에서 실행을 열어 `이미지 만들기와 올리기` 단계의 로그를 봅니

다. Dockerfile의 각 줄이 차례로 실행되고, 마지막에 이미지 이름과 다이제스트가 찍힙니다. 이어서 배포 단계가 끝나면 `배포 결과` 단계에 배포 주소와 배포한 이미지 이름이 나옵니다.

직접 확인할 곳: Actions 실행과 저장소 확인

1. GitHub 저장소 > Actions 탭 > 방금 실행된 deploy > deploy 작업 > `이미지 만들기와 올리기` 단계 펼치기
2. 같은 실행의 `배포 결과` 단계에서 배포 주소와 이미지 이름(끝이 커밋 SHA)을 확인
3. Google Cloud 콘솔 > Artifact Registry > friend-agent > friend-agent: 커밋 SHA 태그가 붙은 이미지 한 개
4. staging 주소의 `/version` : `commit` 이 방금 머지한 커밋 SHA와 같은지 확인

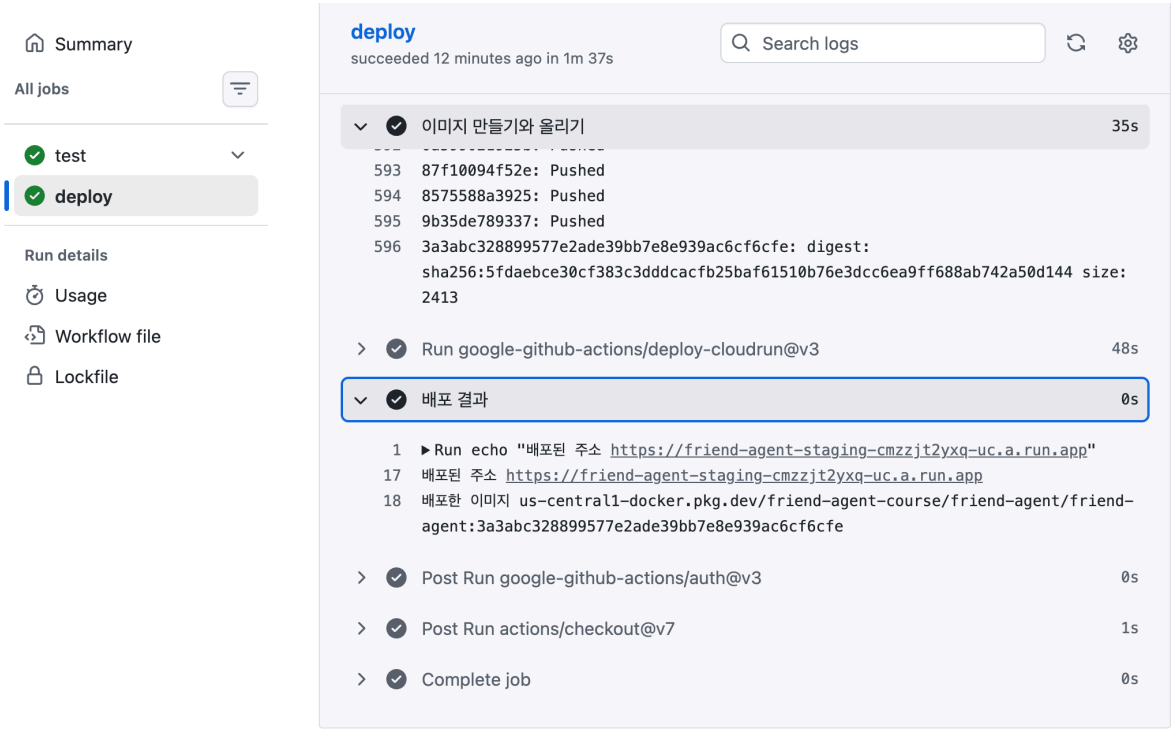


그림: GitHub 저장소 > Actions > deploy 실행 > 이미지 만들기와 올리기 단계: docker build 의 단계 로그와 마지막 docker push 줄의 이미지 이름, 다이제스트

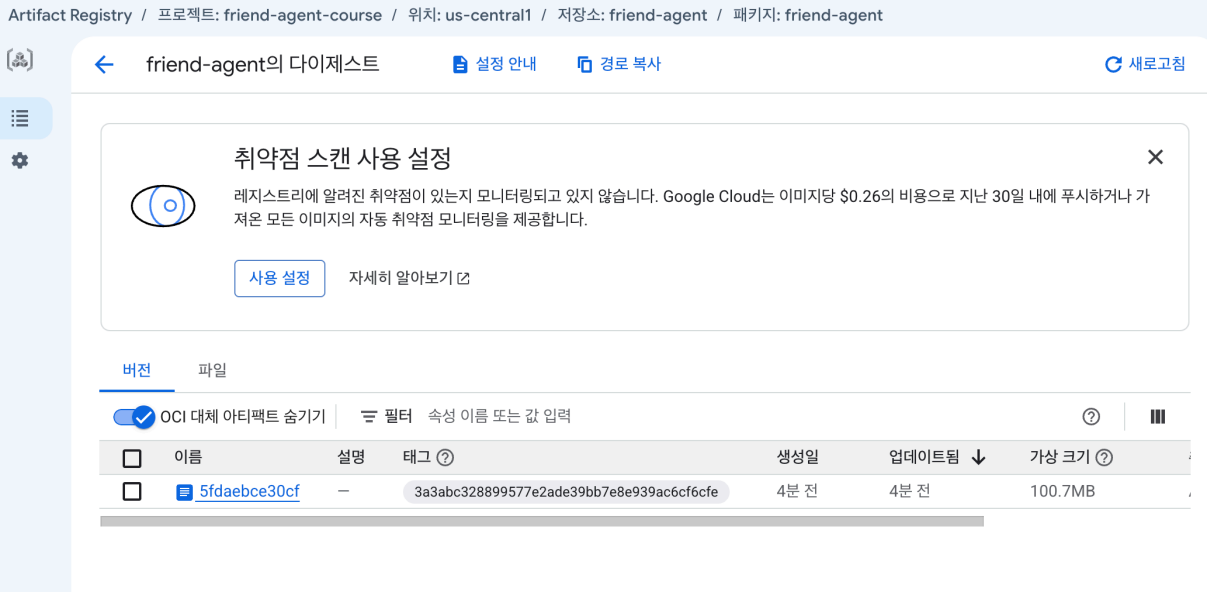


그림: Artifact Registry > friend-agent > friend-agent: 커밋 SHA 태그가 붙은 이미지, 다이제스트 앞부분, 크기, 만든 시각

staging에서 OCR(광학 문자 인식, 이미지 속 글자를 텍스트로 바꾸는 기술) 기능과 대화가 그대로 되는지 확인했으면 staging에서 main으로 PR을 올려 머지하고, production 배포를 승인합니다. 승인 화면에서 실행을 열어 이번 이미지 태그를 다시 보면 staging 때와 커밋 SHA가 다를 수 있습니다. main에 머지하면서 새 커밋이 생기면 그 커밋으로 이미지를 다시 만들기 때문입니다. staging에서 확인한 이미지를 다시 만들지 않고 그대로 production에 올리는 방법은 섹션 4에서 직접 해 봅니다.

이미지 기반 배포가 처음 실행될 때 배포 단계가 권한 오류로 멈출 수도 있습니다. 그때는 오류 메시지에 나온 권한 이름을 확인하고, 그 작업에 필요한 역할 하나만 추가합니다. 오류를 없애려고 Owner나 Editor를 주지 않습니다.

객관식 퀴즈

Q. 이미지 기반 배포로 바꾸는 PR을 검토하고 있습니다. 다음 중 머지하기 전에 반드시 고쳐 달라고 해야 하는 변경은 무엇인가요?

- 1. `docker build --tag "$IMAGE" .` 의 `IMAGE` 가 `AR_REPO/friend-agent:${{ github.sha }}` 이다
- 2. 배포 단계가 `source: .` 대신 `image: ${{ env.IMAGE }}` 를 쓴다
- 3. 빌드 단계에 `--build-arg LLM_API_KEY=${{ secrets.LLM_API_KEY }}` 가 추가되었다
- 4. `setup-gcloud@v3` 단계가 추가되었다

- •

  1. `docker build --tag "$IMAGE" .` 의 `IMAGE` 가 `AR_REPO/friend-agent:${{ github.sha }}` 이다

- •

  2. 배포 단계가 `source: .` 대신 `image: ${{ env.IMAGE }}` 를 쓴다

- •

  3. 빌드 단계에 `--build-arg LLM_API_KEY=${{ secrets.LLM_API_KEY }}` 가 추가되었다

- •

4. `setup-gcloud@v3` 단계가 추가되었다

Chapter04

연결표 채우기

이제 훈련의 질문을 다시 꺼냅니다. 섹션 3에서 배포한 production이 정말 그 커밋으로 돌고 있는지 확인하고, 어떤 커밋이 어떤 이미지와 리비전으로 배포됐는지 잇는 추적 표인 연결표를 채웁니다. 아래 그림의 다섯 단계는 각자 앞 단계의 값을 넘겨받아 기록합니다. Actions는 커밋 SHA로 태그를 붙이고, Artifact Registry는 태그와 다이제스트를 함께 저장하고, Cloud Run 리비전은 배포할 때 이미지의 다이제스트로 고정되고, 서비스는 `COMMIT_SHA` 환경변수로 커밋 SHA를 `/version` 에 돌려줍니다.

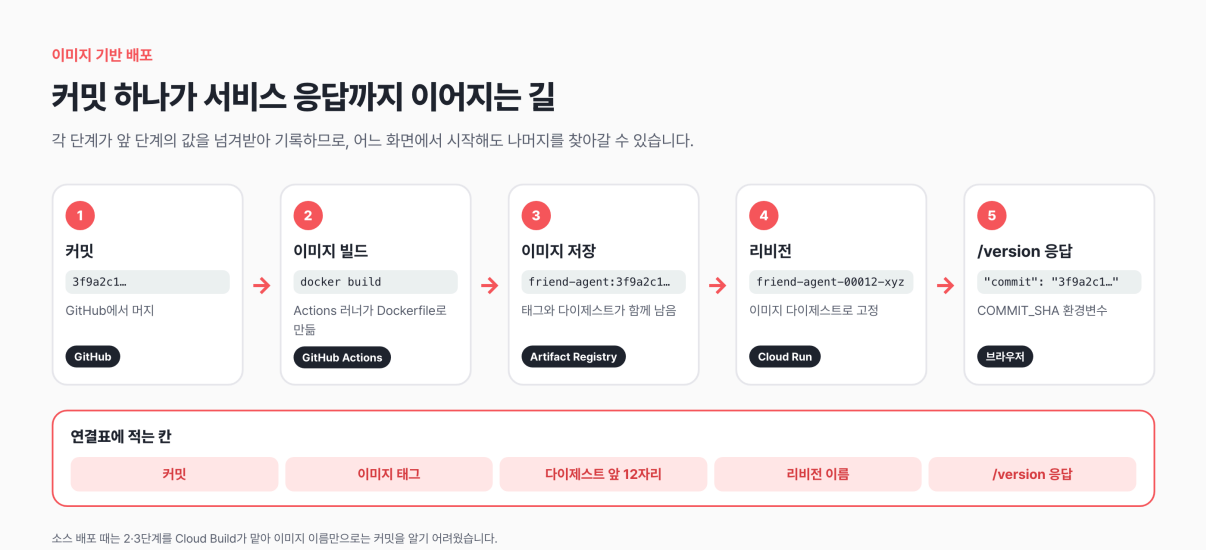


그림: 커밋, 이미지 빌드, 이미지 저장, 리비전, /version 응답으로 이어지는 다섯 단계

다섯 단계의 값을 하나씩 찾아 아래 연결표를 채웁니다. staging과 production을 한 줄씩 채웁니다. 커밋 SHA와 다이제스트는 길어서 앞 12자리만 적어도 구별됩니다.

환경	커밋	이미지 태그	이미지 다이제스트(앞 12자리)	리비전 이름	`/version` 응답의 commit, revision
staging					
production					
production (승격 뒤)					

마지막 줄은 아래 이미지 승격까지 마친 뒤에 채웁니다.

직접 실행할 명령: 연결표의 값 찾기

```
# 1. 커밋: main 의 마지막 커밋
git log -1 --format=%H origin/main

# 2·3. 이미지 태그와 다이제스트
```

```
gcloud artifacts docker images list us-central1-docker.pkg.dev/<PROJECT_ID>/friend-agent/friend-agent --
include-tags

# 4. 지금 요청을 받는 리비전과 그 리비전의 이미지
gcloud run revisions list --service friend-agent --region us-central1
gcloud run revisions describe <리비전 이름> --region us-central1
```

코딩 에이전트에게 맡길 때: 연결표 채우기

```
production 서비스 friend-agent(us-central1)에 대해 다음 값을 찾아 표로 보여 줘.
main 의 마지막 커밋 SHA, 그 커밋 SHA 태그가 붙은 이미지의 다이제스트, 지금 트래픽을 받는 리비전 이름과
그 리비전이 쓰는 이미지. 조회만 하고 아무것도 바꾸지 마. staging(friend-agent-staging)도 같은 방식으로.
```

리비전 상세 출력에서는 이미지 줄을 찾습니다. 워크플로는 커밋 SHA 태그로 배포했지만, Cloud Run은 배포할 때 태그를 그 순간의 다이제스트로 바꿔 리비전에 기록하므로 이미지 주소는 `…/friend-agent@sha256:…` 처럼 다이제스트로만 보입니다. 이 다이제스트를 Artifact Registry 목록에서 찾으면 같은 줄에 커밋 SHA 태그가 붙어 있고, 이렇게 리비전에서 커밋까지 거슬러 올라갑니다. 마지막 칸은 브라우저로 서비스 주소 끝에 `/version` 을 붙여 열어 `commit` 과 `revision` 값을 적습니다. 콘솔에서는 Cloud Run > `friend-agent` > 리비전 탭에서 리비전을 누르면 오른쪽 상세에 이미지 주소가 나옵니다.

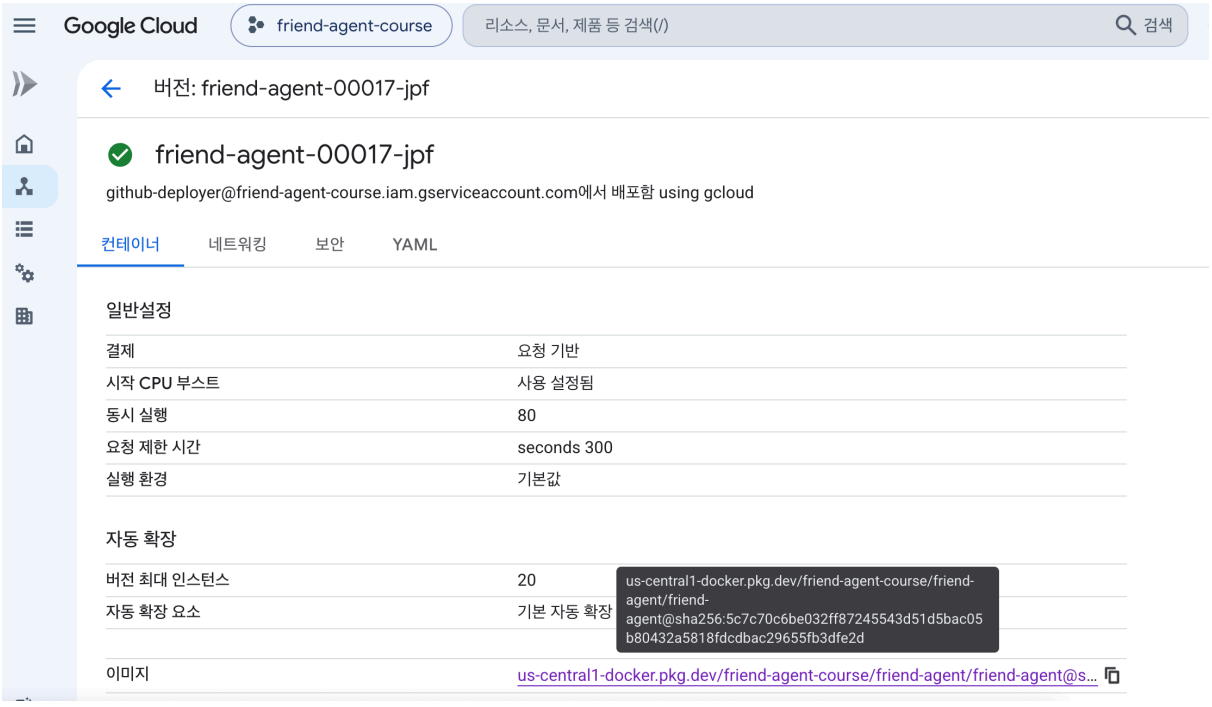


그림: Cloud Run > friend-agent > 리비전 탭 > 트래픽 100% 리비전 > 상세의 컨테이너 항목: 이미지 주소가 friend-agent@sha256:... 처럼 다이제스트로 적힌 모습.

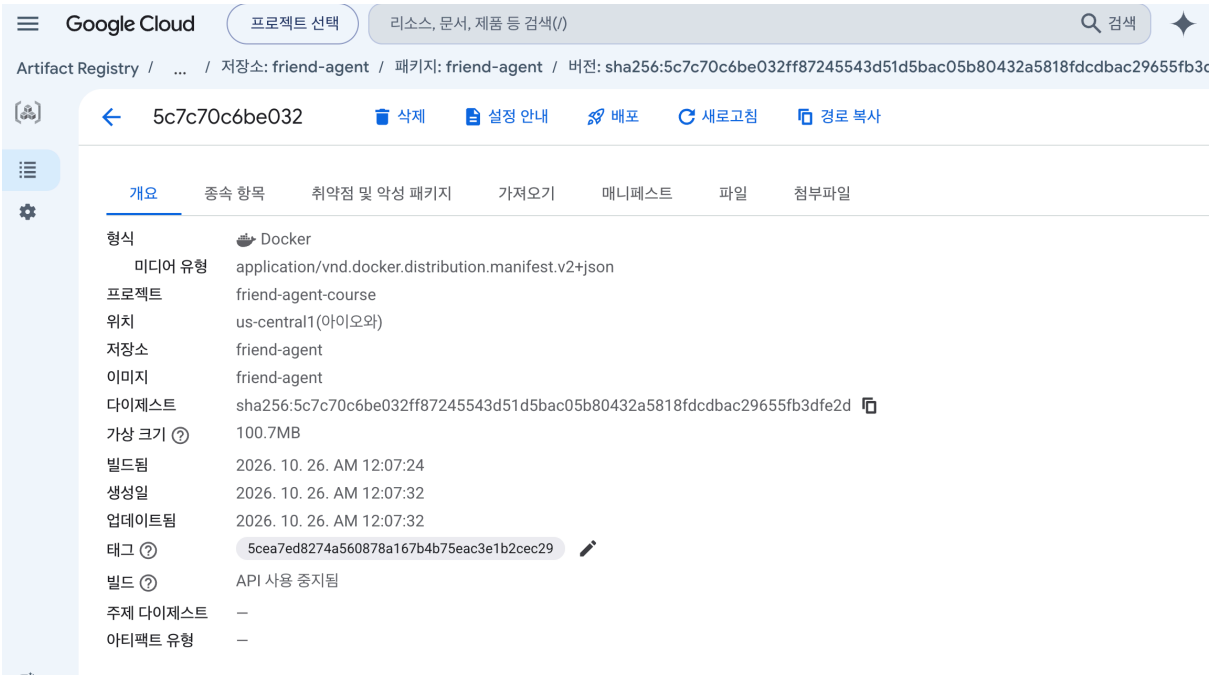


그림: 같은 다이제스트를 Artifact Registry 목록에서 찾으면 커밋 SHA 태그가 붙어 있음

다섯 칸이 모두 같은 커밋을 가리키면 "지금 production에서 도는 코드는 어떤 커밋인가"에 화면 몇 개로 답할 수 있습니다. 거꾸로도 쓸 수 있습니다. 사용자가 문제를 알려 오면 `/version` 의 커밋 SHA에서 출발해 GitHub에서 그 커밋의 변경을 열고, 같은 태그의 이미지와 그 이미지를 쓰는 리비전을 찾아 롤백할 대상을 정합니다.

주관식 퀴즈

Q. [이해 점검] 연결표를 채우다 보니 `/version` 의 commit 값과 리비전이 쓰는 이미지의 태그가 서로 다른 커밋 SHA를 가리킨다고 가정해 보세요. 어떤 경우에 이런 일이 생길 수 있을지, 무엇부터 확인할지 적어 보세요.

0/300**다음**

이미지 승격

연결표의 두 줄을 나란히 보면 staging과 production의 커밋 SHA와 다이제스트가 다르게 적혀 있을 것입니다. staging에서 main으로 머지할 때 머지 커밋이 새로 생겼고, 워크플로가 그 커밋으로 이미지를 다시 만들었기 때문입니다. 코드 내용은 같아도 production에서 도는 이미지는 staging에서 확인한 이미지가 아닙니다. staging에서 확인한 이미지를 다시 빌드하지 않고 그대로 다음 환경에 배포하는 것을 이미지 승격(image promotion)이라고 합니다. 승격에는 태그 대신 다이제스트를 씁니다. 태그는 다른 이미지로 옮겨 붙일 수 있지만, 다이제스트는 이미지 내용으로 정해지므로 확인한 그 이미지 하나만 가리킵니다.

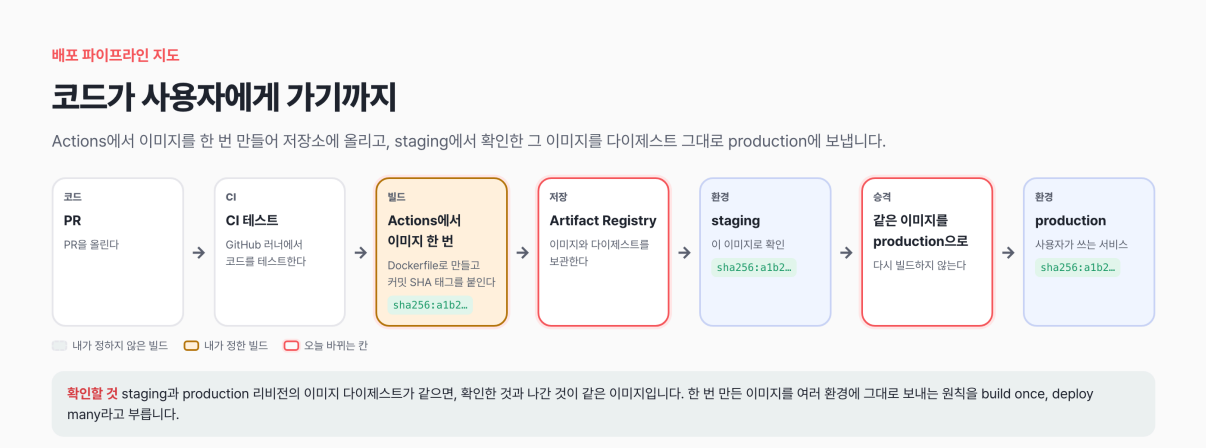


그림: Actions에서 한 번 만든 이미지를 Artifact Registry에 두고, staging에서 확인한 다이제스트 그대로 production에 보내는 파이프라인

승격은 두 번에 나눠 해 봅니다. 먼저 명령 한 줄로 직접 해서 무엇이 바뀌는지 보고, 이어서 같은 일을 배포 워크플로가 하게 옮깁니다. 직접 할 때는 staging에서 트래픽을 받는 리비전의 이미지를 다이제스트까지 붙은 주소로 알아낸 뒤, 그 주소로 production 서비스를 배포합니다. 이때 `COMMIT_SHA` 환경변수도 staging의 커밋 SHA로 함께 바꿉니다. 이미지만 바꾸면 기존 환경변수가 그대로 남아 `/version` 이 main의 머지 커밋을 계속 보여 주고, 연결표의 첫 칸과 마지막 칸이 어긋나기 때문입니다. `ENV`, 시크릿, 서비스 계정 같은 나머지 설정은 production 서비스에 이미 있는 값이 유지됩니다.

직접 실행할 명령: staging 이미지를 production으로 승격하기

```
# 1. staging 에서 트래픽을 받는 리비전과 그 이미지(다이제스트 포함 주소)
gcloud run services describe friend-agent-staging --region us-central1 --format "value(status.latestReadyRevisionName)"
gcloud run revisions describe <staging 리비전 이름> --region us-central1 --format "value(status.imageDigest)"

# 2. staging 의 커밋 SHA: 브라우저로 staging 주소의 /version 을 열어 commit 값을 적는다

# 3. 같은 이미지를 production 에 배포한다. 이미지는 @sha256: 로 끝나는 주소를 그대로 쓴다
gcloud run deploy friend-agent --region us-central1 --image <1에서 얻은 주소> --update-env-vars COMMIT_SHA=<2에서 적은 커밋 SHA>
```

코딩 에이전트에게 맡길 때: 이미지 승격

```
us-central1 의 friend-agent-staging 에서 지금 트래픽을 받는 리비전의 이미지를 다이제스트가 붙은 주소(@sha256:...)로 알려 주고,
staging 리비전의 COMMIT_SHA 환경변수 값도 보여 줘.
내가 확인하면 그 이미지 주소 그대로 production 서비스 friend-agent 를 배포하고, COMMIT_SHA 만 staging 의 값으로 바꿔 줘.
태그로 바꿔 쓰지 말고, 다른 환경변수와 시크릿, 서비스 계정 설정은 건드리지 마.
```

실행 전에 서비스 이름이 `friend-agent` 인지, 이미지 주소가 태그(`:커밋 SHA`)가 아니라 `@sha256:` 으로 끝나는지, 환경변수 옵션이 기존 값을 지우는 `--set-env-vars` 가 아니라 `--update-env-vars` 인지 확인합니다. 이 명령은 GitHub의 배포 승인을 거치지 않고 production을 바꾸므로, staging에서 OCR과 대화를 확인한 뒤에 내가 직접 실행을 승인합니다.

직접 확인할 곳: 승격 결과 확인

1. Cloud Run > `friend-agent` > 리비전 탭에서 새 리비전이 트래픽 100%를 받는지 봅니다.
2. 새 리비전의 컨테이너 항목에서 이미지 다이제스트를 `friend-agent-staging` 의 리비전과 비교합니다.
3. production 주소의 `/version` 을 열어 `commit` 이 staging의 커밋 SHA와 같은지 봅니다.
4. 연결표에 승격 뒤 production 줄을 새로 한 줄 채우고, staging 줄과 다이제스트가 같은지 확인합니다.

다이제스트가 같아졌다면 staging에서 확인한 이미지와 사용자에게 나간 이미지가 같다고 말할 수 있습니다. 섹션 2에서 말한 build once, deploy many가 이것입니다. 다만 워크플로는 아직 그대로라서, 다음에 main으로 머지하면 production용 이미지를 다시 만들어 이 승격을 덮어씁니다. 그래서 방금 손으로 한 일을 워크플로로 옮깁니다.

승격을 워크플로로 옮기기

이제 방금 손으로 한 1~3번을 배포 워크플로가 하게 바꿉니다. 섹션 3에서 바꾼 `.github/workflows/deploy.yml` 을 그대로 두고 세 군데만 고칩니다. 테스트를 먼저 돌리고, 브랜치에 따라 staging이나 production 환경으로 배포하고, production은 승인을 기다리는 구조는 그대로입니다.

고칠 곳	지금	바꾼 뒤
1. 이미지 만들기와 올리기 단계	두 브랜치 모두에서 이미지를 만듦	`staging` 브랜치에서만 실행하고, 만든 이미지 이름과 커밋을 다음 단계로 넘김
2. staging 이미지 찾기 단계 (새로 추가)	없음	`main` 브랜치에서만 실행. staging 서비스에서 트래픽을 가장 많이 받는 리비전의 다이제스트와 `COMMIT_SHA` 를 읽어 다음 단계로 넘김
3. 배포 단계	`image:` 와 `COMMIT_SHA` 가 이번 실행의 커밋으로 정해짐	앞 단계에서 넘겨받은 이미지와 커밋을 씀

고친 뒤의 배포 작업은 아래와 같습니다. `env:` 의 `IMAGE` 줄은 지우고, `steps:` 의 `setup-gcloud` 다음부터를 이렇게 바꿉니다. `if:` 는 그 단계를 어느 브랜치에서 실행할지 정하는 조건이고, `$GITHUB_ENV` 에 적은 값은 같은 작업의 다음 단계에서 `${{ env.이름 }}` 으로 읽을 수 있습니다.

```
env:
STAGING_SERVICE: friend-agent-staging
steps:
# (checkout, auth, setup-gcloud 단계는 그대로)

# 1. staging 브랜치에서만 이미지를 만들고, 이미지 이름과 커밋을 다음 단계로 넘긴다
-name: 이미지 만들기와 올리기 (staging)
if: github.ref_name!= 'main'
run:|
          IMAGE="${{ vars.AR_REPO }}/friend-agent:${{ github.sha }}"
          gcloud auth configure-docker us-central1-docker.pkg.dev --quiet
          docker build --tag "$IMAGE" .
          docker push "$IMAGE"
          echo "IMAGE=$IMAGE" >> "$GITHUB_ENV"
          echo "COMMIT=${{ github.sha }}" >> "$GITHUB_ENV"

# 2. main 브랜치에서는 이미지를 만들지 않고, staging 리비전의 다이제스트와 커밋을 찾는다
-name: staging 이미지 찾기 (production 승격)
if: github.ref_name == 'main'
run:|
          REV=$(gcloud run services describe "$STAGING_SERVICE" --region us-central1 --format json | jq
-r '.status.traffic | max_by(.percent) | .revisionName')
          gcloud run revisions describe "$REV" --region us-central1 --format json > rev.json
          echo "IMAGE=$(jq -r '.status.imageDigest' rev.json)" >> "$GITHUB_ENV"
```

```
          echo "COMMIT=$(jq -r '.spec.containers[0].env[] | select(.name=="COMMIT_SHA") | .value' rev.js
on)" >> "$GITHUB_ENV"
          echo "staging 리비전 $REV 의 이미지를 승격한다"

# 3. 배포 단계는 넘겨받은 이미지와 커밋을 쓴다 (다른 줄은 그대로)
-id: deploy
uses: google-github-actions/deploy-cloudrun@v3
with:
...
image: ${{ env.IMAGE}}
env_vars:|
            COMMIT_SHA=${{ env.COMMIT }}
            ENV=${{ vars.ENV }}
```

이렇게 바꾸면 production 배포 승인의 뜻도 달라집니다. 지금까지는 "main의 코드로 이미지를 새로 만들어 내보낸다"를 승인했다면, 이제는 "staging에서 확인한 그 이미지를 내보낸다"를 승인합니다. 승인을 기다리는 동안에는 워크플로의 단계가 아직 실행되지 않으므로, 승인하기 전에 staging 주소의 `/version` 과 리비전 탭에서 지금 staging이 확인한 그 버전인지 봅니다. staging에서 확인하는 동안 다른 변경이 staging에 머지되면, 승격되는 것은 그 뒤의 이미지가 됩니다.

직접 실행할 명령: 승격 워크플로로 바꾸기

```
git switch staging
git pull
git switch -c promote-workflow
# .github/workflows/deploy.yml 을 위 세 군데처럼 고친 뒤
git diff
git add .github/workflows/deploy.yml
git commit -m "production 은 staging 이미지를 승격해 배포"
git push -u origin promote-workflow
gh pr create --base staging --title "production 은 staging 이미지를 승격" --body "main 머지 때 이미지를 다시 만
들지 않는다"
```

코딩 에이전트에게 맡길 때: 승격 워크플로로 바꾸기

```
staging 에서 promote-workflow 브랜치를 만들고 .github/workflows/deploy.yml 의 deploy 작업을 이렇게 바꿔 줘.
1. 이미지 만들기와 올리기 단계는 main 이 아닌 브랜치에서만 실행하고, 만든 이미지 이름과 github.sha 를 GITHUB_ENV 의 IMAG
E, COMMIT 으로 넘겨 줘.
2. main 브랜치에서만 실행하는 단계를 새로 넣어 줘. staging 서비스 friend-agent-staging 에서 트래픽을 가장 많이 받는 리비
전의 이미지 다이제스트(status.imageDigest)와 그 리비전의 COMMIT_SHA 환경변수 값을 읽어 IMAGE, COMMIT 으로 넘겨 줘.
3. 배포 단계의 image 와 COMMIT_SHA 는 IMAGE, COMMIT 을 쓰게 바꿔 줘.
테스트, 환경, 승인 구조와 액션 버전, 다른 단계는 바꾸지 마. 바뀐 부분을 git diff 로 보여 주고, 내가 확인하면 커밋해서 stagin
g 으로 PR을 올려 줘.
```

차이(diff)에서는 위 표의 세 군데만 바뀌었는지, 이미지 만들기 단계와 새 단계에 서로 반대인 `if:` 조건이 붙었는지, 배포 단계의 다른 줄(`service`, `flags` 등)이 그대로인지 봅니다. PR을 `staging` 에 머지하면 바뀐 워크플로가 staging용 이미지를 만들어 배포하고, 이어서 `staging` 에서 `main` 으로 PR을 올려 머지한 뒤 production 배포를 승인합니다.

직접 확인할 곳: 승격 워크플로 확인

1. GitHub 저장소 > Actions 탭에서 main 의 deploy 실행을 열고, `이미지 만들기와 올리기 (staging)` 단계가 건너뛰어졌고(회색) `staging 이미지 찾기 (production 승격)` 단계가 실행되었는지 봅니다.
2. `staging 이미지 찾기` 단계 로그에 승격한 staging 리비전 이름, 다이제스트, 커밋이 찍혔는지 봅니다.

3. Cloud Run에서 `friend-agent` 와 `friend-agent-staging` 의 트래픽 100% 리비전 다이제스트가 같은지 봅니다.
4. 두 서비스의 `/version` 에서 `commit` 이 같은지 보고, 연결표에 한 줄을 더 채웁니다.

결과 해석: 다시 만들지 않는 production

main 실행에서 이미지 만들기 단계가 건너뛰어졌다면, production 배포는 Dockerfile을 다시 실행하지 않았습니다. 그래서 그사이 새 라이브러리 버전이 나왔어도 production에 들어가지 않고, staging에서 확인한 이미지 그대로 나갑니다. 두 서비스의 다이제스트가 같고 `/version` 의 `commit` 이 같다면, 섹션 2 훈련의 3번 질문에 이제 매번 "같다"고 답할 수 있습니다. 같은 커밋이 main에서는 머지 커밋이라는 다른 SHA를 갖지만, `/version` 은 이미지를 만든 staging 쪽 커밋을 보여 주므로 커밋에서 이미지까지의 연결도 끊기지 않습니다.

객관식 퀴즈

Q. 손으로 승격할 때 `--image` 에 `friend-agent:<staging 커밋 SHA>` 태그 주소를 쓰지 않고 `@sha256:` 다이제스트 주소를 쓰는 이유로 가장 알맞은 것은 무엇인가요?

• 1. 태그 주소로는 Cloud Run에 배포할 수 없기 때문이다
• 2. 태그는 다른 이미지로 옮겨 붙일 수 있지만, 다이제스트는 이미지 내용으로 정해져 staging에서 확인한 이미지 하나만 가리키기 때문이다
• 3. 다이제스트 주소를 쓰면 환경변수가 자동으로 staging 값으로 바뀌기 때문이다
• 4. 다이제스트 주소를 쓰면 다음 main 머지 때 이미지를 다시 만들지 않기 때문이다**제출**

- •

  1. 태그 주소로는 Cloud Run에 배포할 수 없기 때문이다

- •

  2. 태그는 다른 이미지로 옮겨 붙일 수 있지만, 다이제스트는 이미지 내용으로 정해져 staging에서 확인한 이미지 하나만 가리키기 때문이다

- •

  3. 다이제스트 주소를 쓰면 환경변수가 자동으로 staging 값으로 바뀌기 때문이다

- •

  4. 다이제스트 주소를 쓰면 다음 main 머지 때 이미지를 다시 만들지 않기 때문이다

로컬 환경에서 만든 이미지를 올리지 않는 이유

이미지는 로컬 환경(내 컴퓨터)의 Docker로도 만들 수 있습니다. 그러면 Actions를 거치지 않고 로컬 환경에서 만들어 바로 올리면 더 빠르지 않을까요. 이 과정에서는 그렇게 하지 않는데, 이유는 두 가지입니다.

첫째는 CPU 아키텍처입니다. CPU 아키텍처는 amd64나 arm64처럼 CPU가 실행할 수 있는 명령 형식을 말하고, Docker에서는 플랫폼이라고 부르며 `linux/amd64` , `linux/arm64` 처럼 적습니다. Docker는 기본으로 그 컴퓨터의 CPU 아키텍처에 맞춘 이미지를 만듭니다. 그래서 Apple Silicon(M 시리즈) Mac에서 만든 이미지는 arm64용이 되는데, Cloud Run은 Linux amd64용 이미지를 요구하므로 이 이미지는 배포에서 실패합니다. GitHub Actions의 `ubuntu-latest` 러너는 amd64라서 섹션 3의 워크플로가 만든 이미지는 이 문제가 없습니다. Intel 칩 Mac이나 대부분의 Windows 컴퓨터처럼 amd64인 컴퓨터에서는 배포가 되기도 하지만, 둘째 이유는 그대로 남습니다.

아래 사례는 Apple Silicon Mac에서 Dockerfile로 이미지를 만들어 시험용 저장소에 올리고 그 이미지로 시험용 서비스를 배포해 본 결과입니다. 쓴 명령은 섹션 3의 워크플로와 같은 `docker build`, `docker push`, `gcloud run deploy --image` 입니다. 배포는 이미지가 지원하는 CPU 아키텍처에 `linux/amd64` 가 없다는 오류로 실패했고, `docker image inspect` 로 이미지의 CPU 아키텍처를 보면 `arm64` 로 나옵니다.



그림: Apple Silicon Mac 터미널: gcloud run deploy --image 실패 메시지와 docker image inspect 결과: linux/amd64 를 지원하지 않는다는 오류 문장, 그리고 Architecture 값 arm64

직접 해 보고 싶다면

같은 실패를 직접 보고 싶다면 아래 순서로 해 봅니다. 실제 서비스와 섞이지 않도록 시험용 저장소 `local-test` 와 시험용 서비스 `local-test` 를 따로 만들고, 끝나면 둘 다 지웁니다. 서비스는 LLM API를 부르지 않는 페이크 LLM으로 띄우고 로그인 없이 열지 않게 두어 비용과 공개 범위를 막습니다. 아래에서 작업 규칙에 "로컬 환경에서 만든 이미지를 올리지 않는다"를 넣고 나면 코딩 에이전트가 이 실습을 거절할 수 있으므로, 규칙을 넣기 전에 하거나 명령을 직접 실행합니다. Apple Silicon Mac이 아니라면 배포가 성공할 수 있는데, 그 경우에는 시험용 서비스의 `/version` 에 커밋 SHA가 없다는 점을 확인합니다.

직접 실행할 명령: 로컬 환경에서 만든 이미지 올려 보기 (선택)

```
# 시험용 저장소. 실제 friend-agent 저장소와 따로 둔다
gcloud artifacts repositories create local-test --repository-format docker --location us-central1
gcloud auth configure-docker us-central1-docker.pkg.dev

# 저장소 루트에서 로컬 환경의 Docker로 이미지 만들기, CPU 아키텍처 확인, 올리기
docker build -t us-central1-docker.pkg.dev/<PROJECT_ID>/local-test/friend-agent:local .
docker image inspect --format "{{.Architecture}}" us-central1-docker.pkg.dev/<PROJECT_ID>/local-test/fri
end-agent:local
docker push us-central1-docker.pkg.dev/<PROJECT_ID>/local-test/friend-agent:local

# 시험용 서비스로 배포해 보기 (페이크 LLM, 로그인 없이 열지 않음)
gcloud run deploy local-test --region us-central1 --image us-central1-docker.pkg.dev/<PROJECT_ID>/local-
test/friend-agent:local --set-env-vars LLM_PROVIDER=fake --no-allow-unauthenticated

# 확인이 끝나면 시험용 서비스와 저장소를 지운다 (이름이 local-test 인지 다시 확인)
gcloud run services delete local-test --region us-central1
gcloud artifacts repositories delete local-test --location us-central1
```

Apple Silicon Mac에서 `docker build` 에 `--platform linux/amd64` 를 붙이면 amd64용 이미지를 만들 수 있어 배포까지 됩니다. 하지만 그렇게 해도 아래의 둘째 이유는 해결되지 않습니다.

둘째는 기록과 무결성입니다. 로컬 환경의 `docker build` 는 GitHub에 올라간 커밋이 아니라 내 폴더에 지금 있는 파일로 이미지를 만듭니다. 그래서 커밋하지 않은 수정이나 시험 삼아 바꿔 둔 파일이 이미지에 섞여도 알 수 없고, 그 코드는 GitHub 어디에도 남지 않습니다. PR 검토, CI 테스트, staging 확인, 배포 승인도 모두 건너뛰므로, 이 이미지가 검토하고 테스트한 코드와 같다고 보장할 방법이 없습니다. 누가 언제 만들었는지 기록도 없어서 섹션 4의 연결표는 첫 칸부터 끊기고, 문제가 생겨도 같은 이미지를 다시 만들 수 없습니다. 반대로 Actions에서 만든 이미지는 GitHub에 있는 커밋으로만 만들어지고, 그 커밋은 PR과 CI를 거쳤으며, 커밋 SHA 태그로 언제든 그 코드를 다시 찾을 수 있습니다. 그래서 "배포용 이미지는 GitHub Actions에서만 만든다"를 작업 규칙에 넣습니다. 코딩 에이전트가 배포 오류를 빨리 해결하려고 로컬 환경에서 이미지를 만들어 올리자고 제안할 수 있는데, 규칙 파일에 적어 두면 그 제안이 나오기 전에 막을 수 있습니다. 손으로 작업할 때도 같은 규칙을 실행 전 점검 목록으로 씁니다.

직접 확인할 곳: 작업 규칙에 한 줄 추가

1. 저장소 루트의 `AGENTS.md` 를 연다(Claude Code의 `CLAUDE.md` 는 이 파일을 가져와 쓴다)
2. `## 배포와 권한` 아래에 "배포용 이미지는 GitHub Actions에서만 만든다. 로컬 환경에서 만든 이미지를 Artifact Registry에 올리거나 배포하지 않는다."를 추가한다
3. 작업 브랜치에서 커밋하고 staging으로 PR을 올린다

코딩 에이전트에게 맡길 때: 작업 규칙 추가

```
AGENTS.md 의 "배포와 권한" 아래에 다음 규칙을 한 줄 추가해 줘.
"배포용 이미지는 GitHub Actions에서만 만든다. 로컬 환경에서 만든 이미지를 Artifact Registry에 올리거나 배포하지 않는다."
다른 줄은 바꾸지 말고, 작업 브랜치에서 커밋해서 staging 으로 PR을 올려 줘.
```

객관식 퀴즈

Q. 로컬 환경에서 만든 이미지를 Cloud Run에 배포하는 것을 규칙으로 막는 이유로 가장 알맞은 것은 무엇인가요?

• 1. 로컬 환경의 Docker로는 이미지에 태그를 붙일 수 없기 때문이다
• 2. 로컬 환경의 CPU 아키텍처에 맞춘 이미지는 Cloud Run에서 실행되지 않을 수 있고, 어떤 코드로 만들었는지 기록이 남지 않기 때문이다
• 3. 로컬 환경에서 만든 이미지는 Artifact Registry에 올라가지 않기 때문이다
• 4. 로컬 환경에서 만든 이미지는 크기가 항상 더 크기 때문이다

- •

  1. 로컬 환경의 Docker로는 이미지에 태그를 붙일 수 없기 때문이다

- •

  2. 로컬 환경의 CPU 아키텍처에 맞춘 이미지는 Cloud Run에서 실행되지 않을 수 있고, 어떤 코드로 만들었는지 기록이 남지 않기 때문이다

- •

  3. 로컬 환경에서 만든 이미지는 Artifact Registry에 올라가지 않기 때문이다

- •

  4. 로컬 환경에서 만든 이미지는 크기가 항상 더 크기 때문이다

Chapter05

정리 정책 정하기

훈련의 2번 질문에는 "되돌릴 이미지가 아직 남아 있는가"가 들어 있었습니다. 이제 머지할 때마다 `friend-agent` 저장소에 이미지가 하나씩 늘고, 소스 배포로 만든 이미지도 `cloud-run-source-deploy` 에 그대로 남아 있습니다. 섹션 2에서 본 것처럼 무료 저장 공간은 0.5GB라서, 이미지 몇 개면 금방 넘습니다. Artifact Registry의 정리 정책은 조건에 맞는 이미지를 저장소가 스스로 지우게 하는 규칙입니다. 정리 정책에는 조건에 맞는 이미지를 지우는 삭제 정책(delete policy)과 조건에 맞거나 최근 버전인 이미지를 남기는 유지 정책(keep policy)이 있습니다. 두 정책을 함께 걸 수 있고, 두 정책에 동시에 걸리는 이미지는 유지됩니다.



그림: 7일보다 오래된 이미지를 지우되 최근 10개는 남기는 정리 정책

그림의 정책은 "모든 이미지 가운데 7일보다 오래된 것은 지운다"와 "가장 최근 10개는 오래돼도 남긴다"를 함께 건 것입니다. 남기는 수를 정할 때 기준은 롤백입니다. 이전 리비전으로 롤백하려면 그 리비전이 쓰던 이미지가 저장소에 남아 있어야 하므로, 너무 적게 남기면 정작 롤백해야 할 때 이미지가 없습니다. `friend-agent` 저장소에는 staging과 production 이미지가 같은 이름으로 함께 쌓이니, 두 환경을 합쳐 최근 몇 번의 배포까지 롤백할 수 있어야 하는지 생각해 정합니다. 오늘은 10개로 시작합니다.

정책은 바로 적용하지 않고 드라이런(dry run)으로 먼저 겁니다. 드라이런 상태에서는 아무것도 지우지 않고 지웠을 이미지만 기록하며, 콘솔에서는 "테스트 실행"으로 표시됩니다. 정책 변경은 하루쯤 뒤에 반영되고 드라이런 결과도 그 뒤에 감사 로그에서 확인할 수 있습니다(2026년 10월 기준). 그래서 오늘은 드라이런으로 걸어 두고 이미지 목록을 보며 무엇이 지워질지 직접 따져 본 뒤, 다음 수업 전에 실제 적용으로 바꿉니다.

직접 실행할 명령: 정리 정책을 드라이런으로 걸기

```
# docs/cleanup-policy.json 에 아래 두 정책을 저장한다
[
  {"name": "delete-older-than-7d", "action": {"type": "Delete"}, "condition": {"tagState": "any", "older
Than": "7d"}},
  {"name": "keep-recent-10", "action": {"type": "Keep"}, "mostRecentVersions": {"keepCount": 10}}
]

# 두 저장소에 드라이런(--dry-run)으로 건다
gcloud artifacts repositories set-cleanup-policies friend-agent --project <PROJECT_ID> --location us-cen
tral1 --policy docs/cleanup-policy.json --dry-run
gcloud artifacts repositories set-cleanup-policies cloud-run-source-deploy --project <PROJECT_ID> --loca
tion us-central1 --policy docs/cleanup-policy.json --dry-run

# 결과를 확인한 뒤 실제 적용으로 바꿀 때는 --dry-run 대신 --no-dry-run
```

코딩 에이전트에게 맡길 때: 정리 정책 드라이런

```
docs/cleanup-policy.json 에 "7일보다 오래된 이미지는 지우고, 최근 10개는 남긴다" 정책을 만들어 줘.
그 파일로 us-central1 의 friend-agent 저장소와 cloud-run-source-deploy 저장소에 정리 정책을 드라이런(--dry-run)으로
걸어 줘.
실제 삭제가 일어나는 --no-dry-run 은 쓰지 마. 실행 전에 두 저장소 이름과 정책 내용을 다시 보여 줘.
```

실행 전에는 명령에 `--dry-run` 이 들어 있는지, 저장소 이름이 맞는지, 정책 파일에 유지 정책이 함께 있는지를 봅니다. 유지 정책 없이 삭제 정책만 실제로 걸면 7일이 지난 이미지가 모두 지워지고, 그중에는 지금 production이 쓰는 이미지가 들어 있을 수도 있습니다.

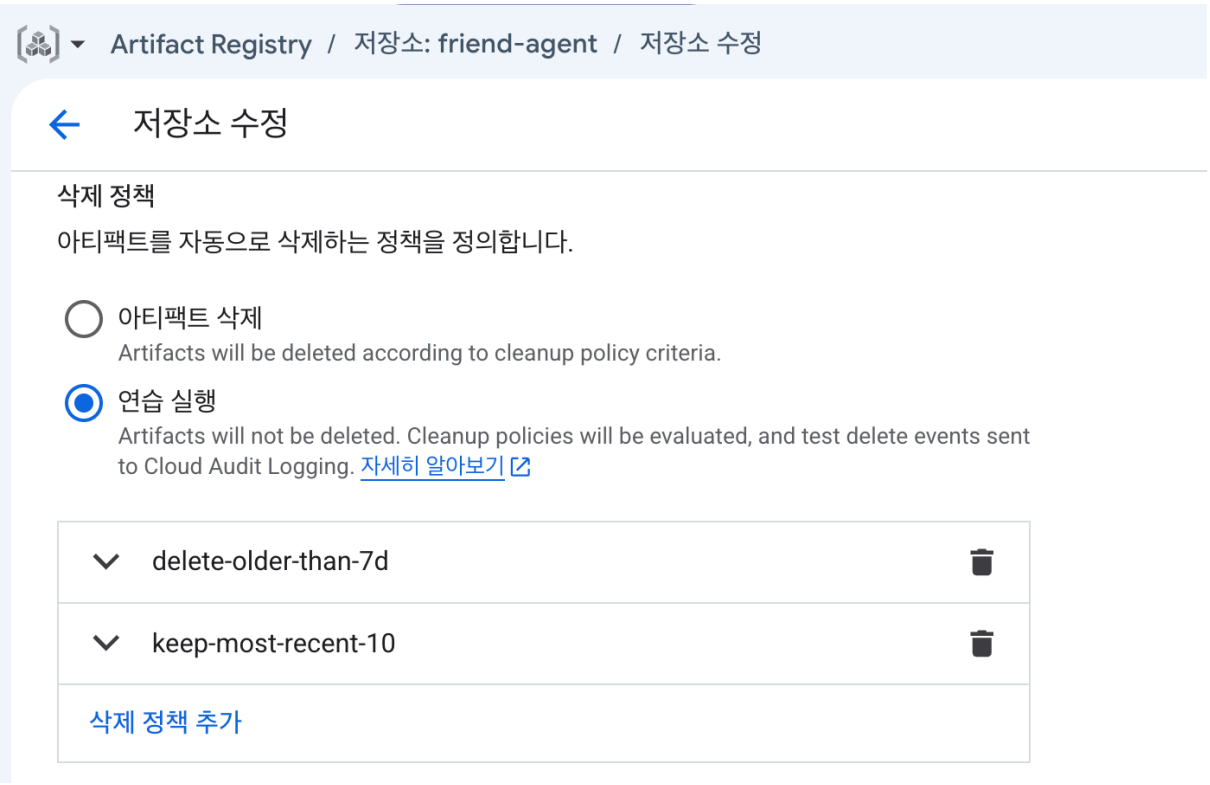


그림: Artifact Registry > friend-agent > 저장소 수정 > 정리 정책: 삭제 정책과 유지 정책 두 개, 그리고 테스트 실행(드라이런)이 켜져 있는 표시

정책을 걸었으면 섹션 2의 목록 명령으로 두 저장소의 이미지를 다시 보고, 만든 시각을 기준으로 어떤 이미지가 지워질지 표시해 봅니다. 섹션 4 연결표의 production 이미지가 남는 쪽에 들어가는지, 바로 전 배포의 이미지도 남는지 확인합니다. 결과가 예상과 다르면 남기는 수를 늘리고 다시 겁니다.

객관식 퀴즈

Q. `friend-agent` 저장소에 이미지가 15개 있고, 그중 6개가 7일보다 오래되었습니다. 그림의 정책(7일보다 오래된 것 지우기, 최근 10개 남기기)을 실제로 적용하면 몇 개가 지워질까요?

- 1. 0개
- 2. 5개
- 3. 6개
- 4. 9개

- •

  1. 0개

- •

  2. 5개

- •

3. 6개

- •

4. 9개

Chapter06

고장 난 Dockerfile 고치기

Dockerfile 한 줄이 바뀌면 이미지에 들어가는 것, 실행하는 사용자, 리슨(listen, 포트를 열고 요청이 들어오기를 기다리는 것)하는 포트가 달라집니다. 템플릿의 `starter/incidents/dockerfiles/` 폴더에는 원래 Dockerfile에서 한 곳씩 바꾼 파일이 네 개 들어 있습니다. 하나씩 저장소 루트에 덮어쓰고, 증상만 보고 무엇이 바뀌었는지 찾은 뒤 되돌립니다. 바뀐 곳을 `git diff` 로 바로 열어 보면 연습이 되지 않으니, 증상과 기록을 먼저 보고 짐작을 적은 다음 마지막에 비교합니다.

문제	덮어쓸 파일	증상을 보는 곳
A	`no-tesseract/Dockerfile` → `Dockerfile`	로컬 컨테이너 또는 staging PR
B	`env-included/dockerignore` → `.dockerignore`	로컬 Docker 필요
C	`root-user/Dockerfile` → `Dockerfile`	로컬 Docker 필요
D	`port-fixed/Dockerfile` → `Dockerfile`	로컬 컨테이너 또는 staging PR

문제마다 같은 순서로 진행합니다.

1. 파일을 덮어쓰고 이미지를 만들어 실행하거나 staging PR로 배포합니다.
2. 무엇이 달라졌는지 증상을 적습니다.
3. 증상이 Dockerfile의 어느 줄과 관련 있을지 짐작해 적고, 원래 파일(`starter/Dockerfile`, `starter/dockerignore`)과 비교해 확인합니다.
4. 원래 파일로 되돌리고 같은 확인을 다시 해 정상인지 봅니다.

코딩 에이전트에게 맡길 때: 원인 찾기 (네 문제 공통)

```
루트의 Dockerfile(또는 .dockerignore)로 만든 이미지에서 아래 증상이 보여. [증상을 적는다]
starter/Dockerfile, starter/dockerignore 와 비교하지 말고, 먼저 증상과 로그를 근거로 어느 줄이 원인일지 설명해 줘. 아직 아무것도 고치지 마.
```

문제 A. OCR만 실패하는 이미지

이미지는 잘 만들어지고 대화도 되지만 다른 기능 하나가 실패합니다. staging에서 보려면 `OCR_ENABLED=true` 가 켜져 있어야 하고, 꺼져 있으면 `gcloud run services update friend-agent-staging --region us-central1 --update-env-vars OCR_ENABLED=true` 로 켭니다.

직접 실행할 명령: 문제 A 적용과 확인

```
cp starter/incidents/dockerfiles/no-tesseract/Dockerfile Dockerfile
docker build -t friend-agent:broken-a .
docker run --rm -p 8080:8080 --env-file .env -e OCR_ENABLED=true friend-agent:broken-a
# 브라우저 http://localhost:8080 에서 📷 버튼으로 data/samples/schedule.png 를 보내 본다
```

```
# 되돌리기
cp starter/Dockerfile Dockerfile
```

▼ **막혔을 때 볼 곳**

컨테이너를 실행한 터미널이나 staging 로그 탭에서 사진을 보낸 시각의 오류 줄을 찾아보세요. 그 줄이 말하는 프로그램은 Python 라이브러리인지, 운영체제에 따로 설치하는 시스템 패키지(apt 패키지)인지 생각해 보고, 시스템 패키지를 설치하는 줄이 Dockerfile 어디에 있었는지 떠올려 보세요.

문제 B. 이미지 안에 들어간 파일

이번에는 Dockerfile이 아니라 이미지에 넣지 않을 파일 목록을 바꿨습니다. 로컬 환경에서 이미지를 만들고 이미지 안의 파일 목록을 봅니다. 이 문제로 만든 이미지는 어디에도 올리지 않고, 확인이 끝나면 바로 지웁니다. 이미지 안에 들어간 파일은 이미지를 받은 누구나 꺼내 볼 수 있습니다.

직접 실행할 명령: 문제 B 적용과 확인

```
cp starter/incidents/dockerfiles/env-included/dockerignore .dockerignore
docker build -t friend-agent:broken-b .
docker run --rm friend-agent:broken-b ls -a /app

# 되돌리기와 이미지 지우기
cp starter/dockerignore .dockerignore
docker image rm friend-agent:broken-b
```

▼ **막혔을 때 볼 곳**

파일 목록에서 저장소에는 커밋되지 않는 파일이 보이는지 찾아보세요. 그 파일이 왜 GitHub Actions에서 만든 이미지에는 들어가지 않고 로컬 환경에서 만든 이미지에만 들어가는지, 두 곳의 폴더에 무엇이 있는지 비교해 보세요.

문제 C. 실행하는 사용자가 바뀐 이미지

서비스는 아무 문제 없이 동작합니다. 겉으로 드러나지 않는 차이를 찾는 문제입니다.

직접 실행할 명령: 문제 C 적용과 확인

```
cp starter/incidents/dockerfiles/root-user/Dockerfile Dockerfile
docker build -t friend-agent:broken-c .
docker run --rm friend-agent:broken-c whoami

# 되돌리기
cp starter/Dockerfile Dockerfile
```

▼ **막혔을 때 볼 곳**

원래 이미지로 같은 명령을 실행했을 때 나오던 이름과 비교해 보세요. 그 이름을 정하는 줄이 Dockerfile 체크리스트의 어느 항목이었는지, 그 항목이 왜 있었는지 다시 읽어 보세요.

문제 D. 접속되지 않는 컨테이너

이미지는 만들어지고 컨테이너도 실행되는데 브라우저로 열리지 않습니다. staging에 배포하면 새 리비전이 시작에 실패하고 트래픽은 이전 리비전에 남습니다.

직접 실행할 명령: 문제 D 적용과 확인

```
cp starter/incidents/dockerfiles/port-fixed/Dockerfile Dockerfile
docker build -t friend-agent:broken-d .
docker run --rm -p 8080:8080 --env-file .env friend-agent:broken-d
# 브라우저로 http://localhost:8080 을 열어 본다. 컨테이너가 남긴 첫 줄도 읽는다

# 되돌리기
cp starter/Dockerfile Dockerfile
```

▼ 막혔을 때 볼 곳

컨테이너가 시작하며 남긴 첫 줄에 어느 주소와 포트를 리슨한다고 적혀 있는지 보세요. `-p 8080:8080` 의 두 숫자가 각각 무엇을 가리키는지, Cloud Run은 앱이 어느 포트를 리슨하기를 기대하고 그 값을 어떻게 알려 주는지 따져 보세요.

네 문제를 풀었으면 `git status` 로 `Dockerfile` 과 `.dockerignore` 가 원래대로 돌아왔는지 확인하고, 로컬 환경에 남은 연습용 이미지는 `docker image rm` 으로 지웁니다.

주관식 퀴즈

Q. [이해 점검] 네 문제 가운데 GitHub Actions에서 만든 이미지로 배포했다면 드러나지 않았을 문제와, 서비스가 정상으로 보여서 그냥 지나쳤을 문제는 각각 어느 것인지 이유와 함께 적어 보세요.

0/300**다음**

선택 과제

남은 시간에는 아래에서 하나 이상을 골라 해 봅니다. 모두 오늘 쓴 명령과 화면으로 할 수 있습니다. 과제 아래에는 무엇을 할지와 다 했는지 확인하는 방법만 적었으니, 필요한 명령은 오늘 실습한 내용을 떠올리거나 코딩 에이전트와 함께 찾아 진행합니다. 실제 서비스나 저장소를 바꾸는 명령은 실행 전에 대상 이름을 다시 확인합니다.

과제	단계	하는 일	다시 확인하는 것
1. 손으로 롤백 훈련	기본	한 커밋 전 이미지로 staging을 되돌렸다가 최신으로 돌려놓는다	섹션 2 훈련의 2번 질문, 태그로 이미지 찾기
2. 태그 덮어쓰기 막기	기본	태그를 바꿀 수 없는 시험용 저장소에 같은 태그로 두 번 올려 본다	태그는 옮겨 붙일 수 있다는 점과 그것을 막는 방법
3. slim 이미지와 전체 이미지 크기 비교	기본	베이스 이미지만 바꿔 로컬 환경에서 만들고 크기를 비교한다	베이스 이미지가 저장 공간과 빌드 시간에 주는 영향
4. 배포 기록 자동으로 남기기	중간	배포가 끝나면 커밋, 리비전, 다이제스트를 Actions 실행 요약에 표로 남긴다	사람이 채우던 연결표를 워크플로가 대신 남기기

과제 1. 손으로 롤백 훈련

섹션 2 훈련의 2번 질문을 끝까지 해 봅니다. Artifact Registry 목록이나 연결표에서 지금보다 한 커밋 전의 커밋 SHA 태그를 찾고, staging을 그 이미지로 배포합니다. 이때 `COMMIT_SHA` 도 그 커밋으로 함께 바꿉니다. 환경변수는 이미지와 따로 남아서, 바꾸지 않으면 `/version` 이 엉뚱한 커밋을 보여 줍니다. staging의

`/version` 이 한 커밋 전을 보여 주는지 확인했으면, 같은 방법으로 최신 커밋의 이미지로 다시 배포해 돌려놓습니다.

다 했는지 확인하는 방법: 되돌린 동안 staging의 `/version` 과 리비전의 다이제스트가 한 커밋 전 이미지의 것과 같고, 돌려놓은 뒤에는 다시 최신 커밋과 같다. 걸린 시간을 섹션 2 훈련 표의 2번 줄 옆에 적는다.

과제 2. 태그 덮어쓰기 막기

커밋 SHA 태그가 믿을 만하려면 같은 태그가 다른 이미지로 옮겨 가지 않아야 합니다. Artifact Registry에는 태그를 바꾸거나 지울 수 없게 하는 태그 변경 금지(immutable tags) 설정이 있습니다. 이 설정은 태그가 붙은 이미지를 지우는 것도 막아 섹션 5의 정리 정책과 부딪힐 수 있으므로, 실제 저장소 `friend-agent` 가 아니라 시험용 저장소 `tag-test` 에서 해 봅니다. 작은 공식 이미지 두 개(예: `busybox`, `alpine`)에 같은 태그를 붙여 차례로 올리고, 두 번째 올리기가 거절되는지 봅니다. 확인이 끝나면 시험용 저장소를 지웁니다.

다 했는지 확인하는 방법: 두 번째 `docker push` 가 태그를 바꿀 수 없다는 오류로 실패하고, 실제 저장소에 이 설정을 켜면 정리 정책과 어떤 점이 부딪히는지 한 문장으로 설명할 수 있다.

과제 3. slim 이미지와 전체 이미지 크기 비교

Dockerfile의 베이스 이미지(base image, Dockerfile이 출발점으로 삼는 이미지)를 `python:3.13-slim-bookworm` 에서 `python:3.13-bookworm` 으로 바꿔 로컬 환경에서 이미지를 만들고, 원래 이미지와 크기를 나란히 봅니다. slim은 실행에 꼭 필요한 파일만 남긴 베이스 이미지입니다. 확인이 끝나면 Dockerfile을 원래대로 되돌리고, 이 비교용 이미지는 올리지 않습니다.

다 했는지 확인하는 방법: 두 이미지의 크기 차이를 적고, 그 차이가 저장 공간(무료 0.5GB)과 배포할 때 이미지를 내려받는 시간에 어떤 영향을 주는지 한 줄로 설명할 수 있다.

과제 4. 배포 기록 자동으로 남기기

섹션 4의 연결표는 배포할 때마다 사람이 채워야 했습니다. GitHub Actions는 실행 화면 맨 위에 실행 요약(job summary)을 보여 주는데, 단계에서 `$GITHUB_STEP_SUMMARY` 파일에 마크다운을 적으면 그 내용이 요약에 나타납니다. 배포 워크플로의 맨 마지막에 단계를 하나 더해, 배포가 끝날 때마다 서비스, 커밋, 리비전, 이미지 다이제스트가 표로 남게 합니다. 리비전 이름과 다이제스트는 배포가 끝난 뒤 Cloud Run에서 조회해 얻습니다. 작업 브랜치에서 고쳐 staging으로 PR을 올리고, 머지된 뒤 Actions 실행 화면을 엽니다.

다 했는지 확인하는 방법: staging과 production 배포 실행의 요약에 표가 생기고, 표의 리비전과 다이제스트가 Cloud Run 리비전 탭의 값과 같다.

📚 이미지 다루기 참고 자료

- Docker 빌드 문서: Multi-stage builds: 단계를 나눠 실행에 필요한 파일만 최종 이미지에 남기는 방법을 예제로 볼 수 있습니다.
- Docker 빌드 문서: Build cache: Dockerfile 줄 순서가 캐시에 어떤 영향을 주는지 설명합니다.
- deploy-cloudrun 액션: `image` 입력과 태그, 트래픽 관련 입력을 확인할 수 있습니다. 버전은 템플릿과 같은 v3를 씁니다.

Chapter07

장애 대응 훈련 다시 하기

섹션 2와 같은 상황, 같은 세 질문으로 훈련을 한 번 더 합니다. 이번에는 production의 `/version`, 리비전 상세의 이미지 주소, Artifact Registry `friend-agent` 의 태그만 보고 답합니다. 섹션 2의 표 옆에 이번 답과 걸린 시간을 적어 나란히 놓습니다. 1번은 `/version` 의 커밋과 이미지 태그가 같은 값이라 바로 답이 나오고, 2번은 연결표와 정리 정책의 유지 규칙으로 바로 전 이미지가 남아 있는지까지 확인할 수 있으며, 3번은 승격한 뒤라면 두 환경의 다이제스트가 같습니다. 승격 워크플로로 바꾼 뒤에는 main에 머지할 때마다 이 답이 "같다"로 유지됩니다.

오늘 배운 것

이미지 이름. 이미지 이름은 저장 위치, 프로젝트, 저장소, 이미지 이름과 태그로 되어 있습니다. 태그는 옮겨 붙일 수 있는 이름이고 다이제스트는 내용으로 정해지는 값이라, 사람이 찾아갈 때는 커밋 SHA 태그를 쓰고 실제로 무엇이 실행 중인지는 다이제스트로 확인합니다.

커밋에서 리비전까지. 이미지를 Actions에서 만들고 커밋 SHA를 태그로 붙이자, 커밋, 이미지, 리비전, `/version` 응답이 같은 값으로 이어졌습니다. 연결표 다섯 칸을 채우면 "지금 도는 코드는 어떤 커밋인가"에 화면 몇 개로 답할 수 있습니다.

이미지 승격. main에 머지하면서 다시 만든 production 이미지는 staging에서 확인한 이미지와 다이제스트가 달랐습니다. 먼저 staging 리비전의 이미지를 다이제스트 주소 그대로 production에 배포해 두 환경의 다이제스트를 맞췄고, 이어서 배포 워크플로가 main에서는 이미지를 만들지 않고 staging의 이미지를 승격하게 바꿨습니다.

실패에서 배운 것. 로컬 환경에서 만든 이미지는 CPU 아키텍처가 달라 Cloud Run에서 실행되지 않았고, 만든 사람과 코드의 기록도 남지 않았습니다. 그래서 배포용 이미지는 Actions에서만 만든다는 규칙을 작업 규칙 파일에 넣었습니다. 이미지 기반 배포에서 권한 오류가 났다면 오류에 나온 작업에 필요한 역할 하나만 더했습니다.

저장 공간. 이미지는 배포할 때마다 쌓이고 무료 저장 공간은 0.5GB입니다. 정리 정책은 드라이런으로 먼저 걸고, 롤백에 쓸 이미지가 남는지 따져 남기는 수를 정했습니다.

직접 고친 것. 고장 난 Dockerfile 네 개에서 증상이 드러나는 곳이 서로 달랐습니다. 기능 하나만 실패하는 이미지, 로컬 환경에서 만든 이미지에만 생기는 문제, 겉으로는 정상인 문제, 시작조차 하지 않는 문제를 증상과 기록에서 출발해 줄 하나로 좁혔습니다.

주관식 퀴즈

Q. [회고] 섹션 2와 방금 한 장애 대응 훈련의 표를 나란히 놓고, 세 질문 가운데 걸린 시간이 가장 많이 줄어든 질문과 그 이유를 적어 보세요. 그리고 오늘 바꾼 구조로도 여전히 사람이 직접 해야 하는 일이 무엇인지 하나 적어 보세요.

0/300**다음**

📚 오늘 나온 자료 모아보기

- Artifact Registry 이미지 이름: 저장소와 이미지 주소의 형식을 확인할 수 있습니다.
- Artifact Registry 이미지 올리기와 내려받기: `gcloud auth configure-docker` 로 인증하고 이미지를 올리는 절차입니다.
- Artifact Registry 정리 정책: 정책 파일 형식, 드라이런 동작, 유지 정책이 우선한다는 규칙이 나옵니다.
- Artifact Registry 요금: 무료 저장 공간 0.5GB와 그 뒤의 요금을 확인하는 곳입니다.

- Cloud Run 컨테이너 런타임 계약: Cloud Run이 요구하는 실행 파일 형식(Linux 64비트 amd64)과 PORT 규칙이 적혀 있습니다.
- Cloud Run 롤백과 트래픽 전환: 이전 리비전으로 롤백하는 방법입니다. 그 리비전의 이미지가 남아 있어야 하는 이유와 이어집니다.

마지막으로

오늘부터 배포되는 이미지는 커밋 SHA로 찾아갈 수 있고, 저장소에는 정리 규칙이 걸려 있습니다. 그래도 부족한 곳이 남아 있습니다. production은 이제 staging에서 확인한 이미지를 그대로 받지만, staging에서 확인하는 동안 다른 변경이 staging에 머지되면 확인하지 않은 이미지가 승격될 수 있어서 승인 전에 staging의 버전을 사람이 다시 봐야 합니다. 이전 버전으로 되돌릴 때도 아직 리비전이나 이미지를 직접 찾아야 합니다. 정리 정책은 아직 드라이런 상태라 실제로 무엇이 지워지는지는 하루 뒤에야 확인할 수 있습니다. 연결표는 배포할 때마다 사람이 채워야 하고, 새 리비전으로 바뀌는 순간 사용자가 오류를 보는지는 아직 측정해 보지 않았습니다. 아침에는 이미지 목록을 보고 짐작만 했다면, 지금은 이 부족한 점들을 태그, 다이제스트, 리비전, 정리 정책 같은 이름으로 짚을 수 있고, 그래서 다음에 무엇을 고칠지 구체적으로 정할 수 있습니다.